Тренажёр игры в палочки

Каждый год студенты и преподаватели собираются на даче за городом, чтобы отдохнуть от работы и поиграть в различные игры. Особую популярность приобрела игра в палочки. Чтобы подготовиться к соревнованию, требуется приложение для тренировки — об этом и задание. Описание игры: Имеется n палочек, расположенных в ряд (например: ||||||). Играют двое по очереди: за ход игрок берёт палочки по определённым правилам. Кто не может сделать ход — проигрывает.

Варианты правил (режимы игры):

1. Стандартно: За ход можно взять от 1 до k (включительно) любых палочек. Брать можно любые палочки (не обязательно подряд).
2. Интервальный выбор: За ход можно взять от a до b (включительно) любых палочек.
3. Подряд: За ход можно взять от 1 до k подряд идущих палочек (никаких пропусков, только связный фрагмент, например, если палочки лежат как ||| ||, то взять 4 палочки не получится: тут изображено пустое пространство после третьей палочки, то есть палочку после третьей взяли на одном из предыдущих ходов).
4. Подряд и интервально: За ход можно взять от a до b подряд идущих палочек.
5. Особое: За ход разрешается взять либо 3 подряд идущих палочки, либо 1 любую палочку, либо 2 любые палочки. Именно в эту версию обычно на даче и играют!

Ограничения на параметры:

- $5 \leq n \leq 50$ — общее количество палочек в игре. Вы можете реализовать решение для меньшего максимального n.
- $1 \leq k \leq n$ — максимальное количество палочек, которое можно взять за ход (для соответствующих режимов).
- $2 \leq a \leq b \leq n$ — минимальное и максимальное количество палочек, которое можно взять за ход (для режимов с диапазоном). Требования к приложению:
- Возможность игры против компьютера.
- Возможность выбора режима и параметров игры при запуске: n, соответствующих режиму параметров (k, a, b), а также, кто ходит первым (игрок или компьютер).
- Обязательное условие: В режиме 1 компьютер должен играть оптимально (где возможно — выигрывать в выигрышных сценариях). Остальные варианты правил (2–5) оцениваются только при наличии реализации 1 варианта и выполнении этого обязательного условия.

За реализацию дополнительных возможностей (например, графический интерфейс, обучающие режимы, советы по оптимальной стратегии, строгое доказательство оптимальности стратегии и т. п.) можно получить бонусные баллы.

1. РАЗРАБОТКА ПРИЛОЖЕНИЯ

В начале разработки были выделены 5 основных этапов:

- планирование;
- проектирование;
- разработка;
- тестирование;
- документирование системы.

1) Планирование:

1. Для реализации программы был выбран язык программирования python для скорейшей разработки приложения.
2. Для создания графического интерфейса была выбрана библиотека PyQt5 как библиотека для создания графических пользовательских интерфейсов из-за простоты и скорости разработки.

2) Проектирование:

Всего было выделено 2 компонента в системе – графический интерфейс и внутренняя логика системы (backend):

Интерфейс:

Приветственный экран:

Кнопка “Play”:

Экран после нажатия “Play”:

Игровые элементы (палочки, в виде кнопок QPushButton с определенным дизайном);

Кнопка остановки (в левом верхнем, срабатывает как при непосредственном нажатии на нее, так и при нажатии на esc):

Экран после нажатия кнопки остановки:

Кнопка “Continue”;

Кнопка “Exit”;

Экран после завершения игры:

QLabel с номером победителя;

Игровое поле (палочки);

Кнопка “Quit”

Кнопка “Settings”:

Экран после нажатия “Settings”:

QCheckBox с названием “Play against the computer”;

QCheckBox с названием “Computer’s turn first”;

Текст ”Level of difficulty”, ниже QComboBox (disabled и текст выше серый если предыдущая кнопка не нажата) с выбором уровня сложности (от 1 до 5);

QRadioButtons (квадратные как QCheckBox) с правилами игры (ее режимами);

Текст “Amount of the sticks”, после этого QSlider от нижней до верхней границы;

Текст “Max amount of sticks for one pick”, после этого QSlider от нижней до верхней границы;

Текст “Min amount of sticks in for one pick in a row”, после этого QSlider от нижней до верхней границы;

Текст “Max amount of sticks in for one pick in a row”, после этого QSlider от значения предыдущего до верхней границы.

Кнопка “Info”;

Кнопка “Exit”;

Backend:

Логика игры для разных режимов;
Сохранение информации из настроек;
Игра против компьютера (для разных режимов);

3) Разработка:

- Для большей гибкости системы и возможности дальнейшей разработки было решено использовать парадигму программирования ООП как мощный инструмент, позволяющий легко модернизировать уже готовые элементы и добавлять новые;

- Таким образом, выла выделена следующая структура проекта:

```
- __pycache__
- data
    - data.json
- modules
    - __pycache__
    - computer_modes
            - __pycache__
        - rule1.py
        - rule2.py
        - rule3.py
    - package
            - __pycache__

            - __init__.py
    - screens
        - __pycache__
        - endscreen.py
        - greetscreen.py
        - pausescreen.py
        - playscreen.py
        - screen.py
```

- settingsscreen.py
  - application.py
  - widgets.py
- style
  - stickbutton.css
- main.py
- README.md
- requirements.txt

- В ходе разработки также была использована распределенная система управления версиями git (ссылка на репозиторий: https://github.com/Vaffi528/Game.git)

- Режимы игры для компьютера:
  (r - кол-во оставшихся палочек)
  1. Режим 1:

<u>Стратегия:</u>

Чтобы выиграть в этом режиме, нужно, чтобы на каждом ходе противника оставалось p(k+1) палочек, где p € N. Для этого компьютер каждый ход должен брать кол-во палочек, равное r mod (k+1) если r mod (k+1) ≠ 0, иначе 100% победу гарантировать нельзя (при оптимальной игре пользователя), поэтому можно брать кол-во палочек [1; k].

<u>Доказательство оптимальности стратегии:</u>

Рассмотрим игру по ходам в обратном порядке:

r = k+1. Как бы ни походил игрок, количество палочек на следующем ходе будет [1; k], а значит, компьютеру нужно будет сделать единственный ход для победы;

Чтобы на последнем ходе было r = k+1 палочек, нужно, чтобы на предыдущем было r = [k+2 (=k+1+1); 2k+1 (=k+1+k)];

Чтобы на предпоследнем ходе было r = [k+2; 2k+1] палочек, нужно, чтобы на предыдущем было r = 2k + 2;

Далее, чтобы было r = 2k + 2 палочек, нужно, чтобы на предыдущем ходе было r = [2k+3; 3k+2];

Далее, чтобы было r = [2k+3; 3k+2] палочек, нужно, чтобы на предыдущем ходе было r = 3k+3;

И т.д.

То есть, чтобы компьютер победил, нужно, чтобы на каждом ходе игрока было p(k+1) палочек, и тогда после его ходя всегда останется от (p-1)k + p до pk + (p-1) палочек, и, если мы возьмем кол-во палочек, равное r mod (k+1), то всегда получим оставшиеся (p-1)(k+1) палочек:

(p-1)k + p – ((p-1)k + p) mod (k+1) =
(p-1)k + p – (p(1+k) - k) mod (k+1) =
(p-1)k + p – (-k) mod (k+1) =
(p-1)k + p – (k + 1 - k) mod (k+1) =
(p-1)k + p – 1 = (p-1)(k+1);
…
pk + (p-1) – (pk + (p-1)) mod (k+1) =
pk + (p-1) – (p(k+1) - 1) mod (k+1) =
pk + (p-1) – (-1) mod (k+1) =
pk + (p-1) – (k + 1 - 1) mod (k+1) =
pk + (p-1) – k = (p-1)(k+1).

2. Режим 2:

<u>Стратегия:</u>

Чтобы гарантированно выиграть в этом режиме, нужно, чтобы на каждом ходе противника оставалось p(b+a) палочек, где p € N. Для этого компьютер каждый ход должен брать кол-во палочек, равное r mod (b+a) если a <= r mod (b+a) <= b, (если < a -> a палочек, если >b -> b, чтобы игрок своим ходом не смог сделать кол-во палочек кратным b + a) иначе 100% победу гарантировать нельзя (при оптимальной игре пользователя).

<u>Доказательство оптимальности стратегии:</u>

Рассмотрим игру по ходам в обратном порядке:

$r = a+b$. Как бы ни походил игрок, количество палочек на следующем ходе будет [a; b], а значит, компьютеру нужно будет сделать единственный ход для победы;

Чтобы на последнем ходе было $r = a+b$ палочек, нужно, чтобы на предыдущем было $r = [b+2a\ (=b+a+a);\ 2b+a\ (=b+a+b)]$;

Чтобы на предпоследнем ходе было $r = [b+2a;\ 2b+a]$; палочек, нужно, чтобы на предыдущем было $r = 2b + 2a$;

Далее, чтобы было $r = 2b + 2a$ палочек, нужно, чтобы на предыдущем ходе было $r = [2b+3a;\ 3b+2a]$;

Далее, чтобы было $r = [2b+3a;\ 3b+2a]$ палочек, нужно, чтобы на предыдущем ходе было $r = 3b+3a$;

И т.д.

То есть, чтобы компьютер победил, нужно, чтобы на каждом ходе игрока было $p(b+a)$ палочек, и тогда после его ходя всегда останется от $(p-1)b + pa$ до $pb + (p-1)a$ палочек, и, если мы возьмем кол-во палочек, равное $r \bmod (b+a)$, то всегда получим оставшиеся $(p-1)(b+a)$ палочек:

$(p-1)b + pa – ((p-1)b + pa) \bmod (b+a) =$
$(p-1)b + pa – (p(b+a) - b) \bmod (b+a) =$
$(p-1)b + pa – (-b) \bmod (b+a) =$
$(p-1)b + pa – (b+a - b) \bmod (b+a) =$
$(p-1)b + pa – a = (p-1)(b+a)$;
…
$pb + (p-1)a – (pb + (p-1)a) \bmod (b+a) =$
$pb + (p-1)a – (p(b+a) - a) \bmod (b+a) =$
$pb + (p-1)a – (-a) \bmod (b+a) =$
$pb + (p-1)a – (b + a - a) \bmod (b+a) =$
$pb + (p-1)a – b = (p-1)(b+a)$.

При этом могут быть случаи, когда r mod (b+a) < a и r mod (b+a) > b. В этих случая нужно соответственно брать a или b, чтобы игрок не мог сделать r кратным a+b:

Представим r как k(a+b) + p, p = r mod (b+a), p < a, p ≠ 0.

Тогда компьютер берет a палочек:

$$r = k(a+b) + p – a = (k-1)a + kb + p$$

Теперь докажем, что что бы ни взял пользоавтель, он не сможет сделать r кратным (b+a):

Пусть он взял q палочек, так, что r кратно (b+a), т.е. kb + p – q = (k-1)b => kb + p – q = kb – b => p – q = -b => q – p = b => q = b + p. Т.к. 0 < p < a, q > b, но это *невозможно*, т.к. максимально возможный ход – b;

Теперь рассмотрим случай, где p > b:

Компьютер берет b палочек:

$$r = k(a+b) + p – b = ka + (k-1)b + p$$

Теперь докажем, что что бы ни взял пользоавтель, он не сможет сделать r кратным (b+a):

Пусть он взял q палочек, так, что r кратно (b+a), т.е. ka + p – q = (k-1)a => ka + p – q = ka – a => p – q = -a => q – p = a => q = a + p. Т.к. b < p < b + a, q > b, но это *невозможно*, т.к. максимально возможный ход – b.

3. Режим 3:
   Чтобы гарантированно выиграть при условии начала игры (компьютер ходит первым), нужно разделить поле на 2 равных куска 1-ым ходом (взяв из центра от 1 до k в зависимости от четности n и k), а далее симметрично повторять действяи противника (т.е. если он выбрал q палочек на одном из участков игрового поля, нужно выбрать столько же и на том же месте (относительно центра, т.е. зеркально) на другом участке). Пример

   | | | | | | | | |

```
|||| ||||
|  || ||||
|  || ||  |
```
и т.д.

Если первым ходит игрок, то гарантированно выиграть не получится (например, если он использует описанную выше стратегию). Можно выиграть, если своим первым ходом он выбрал от 1 до k крайних палочек (тогда можно применить описанную выше стратегию для оставшейся кучи), или если игрок своим ходом взял часть центральных палочек (одну если n нечетное и от 1 до 2 если четное) (тогда можно взять [1; k] палочек после центральных, установив симметрию, а потом применить описанную выше стратегию), или если сделал несимметричный ход.

<u>Доказательство оптимальности стратегии:</u>

Компьютер ходит первым и берет от 1 до k центральных палочек:

пример для n = 10 и k = 5: | | | | | | | | | | -> | | | | _ _ | | | | или | | | _ _ _ _ | | |;

пример для n = 10 и k = 6: | | | | | | | | | | -> | | | | _ _ | | | | или | | | _ _ _ _ | | | или | | _ _ _ _ _ _ | |;

пример для n = 11 и k = 5: | | | | | | | | | | -> | | | | | _ | | | | | или | | | | _ _ _ | | | | или | | | _ _ _ _ _ | | |;

пример для n = 11 и k = 6: | | | | | | | | | | -> | | | | | _ | | | | | или | | | | _ _ _ | | | | или | | | _ _ _ _ _ | | |;

Далее, если он будет зеркально повторять действия пользователя, то он всегда будет уравнивать кол-во палочек (при этом нету разницы, как они расположены, т.к. компьютер просто повторяет за пользователем). Значит, с каждым ходом пользователя, на 1 из

сторон будет всегда больше палочек, а значит, компьютер всегда будет забирать последнюю.

Если же пользователь ходит первым, используя эту стратегию, то компьютер точно проиграет.

Если же пользователь не разобьет своим первым ходом игровое поле на 2 равные кучи (или не будет соблюдать симметрию), то возможно несколько сценариев событий:

1) Пользователь возьмет от 1 до k крайних палочек: В этом случае будет следующая картина: _ _ _ . . . | . . . |
   Тогда, если компьютер разобьет оставшуюся кучу на 2 равных, то мы вернемся к первой стратегии, а значит, компьютер победит.
2) Пользователь возьмет часть центральных палочек, при этом разделив поле пополам симметрии не будет: Тогда мы сможем установить симметрию, взяв необходимые палочки на другой стороне:

(красным помечены позиции, взятые компьютером, желтым – центральные)

пример для n = 10 и k = 6: | | | | | | | | | | -> | | | _ _ _ | | | |.
Тут мы возьмем 1 палочку, получив при этом симметрию:
| | | _ _ _ _ | | |

пример для n = 14 и k = 5: | | | | | | | | | | | | | | ->
| | _ _ _ _ _ | | | | | | | .
Тут мы возьмем 5 палочек, получив при этом симметрию:
| | _ _ _ _ _ _ _ _ _ _ | |.

Таким образом, задача опять сводится к тому, что компьютер будет симметрично копировать действия пользователя.

3) Пользователь не будет соблюдать симметрию. Тогда, если он не разобьет 1 из куч на более мелкие, компьютер может выровнять симметрию (но не во всех случаях):

(красным помечены позиции, взятые компьютером, желтым – позиции, взятые игроком)

пример для n = 14 и k = 5: | | | | | | _ _ | | | | | | ->

| | | | | | _ _ | | _ _ | | -> | _ _ _ | | _ _ | | _ _ | |.

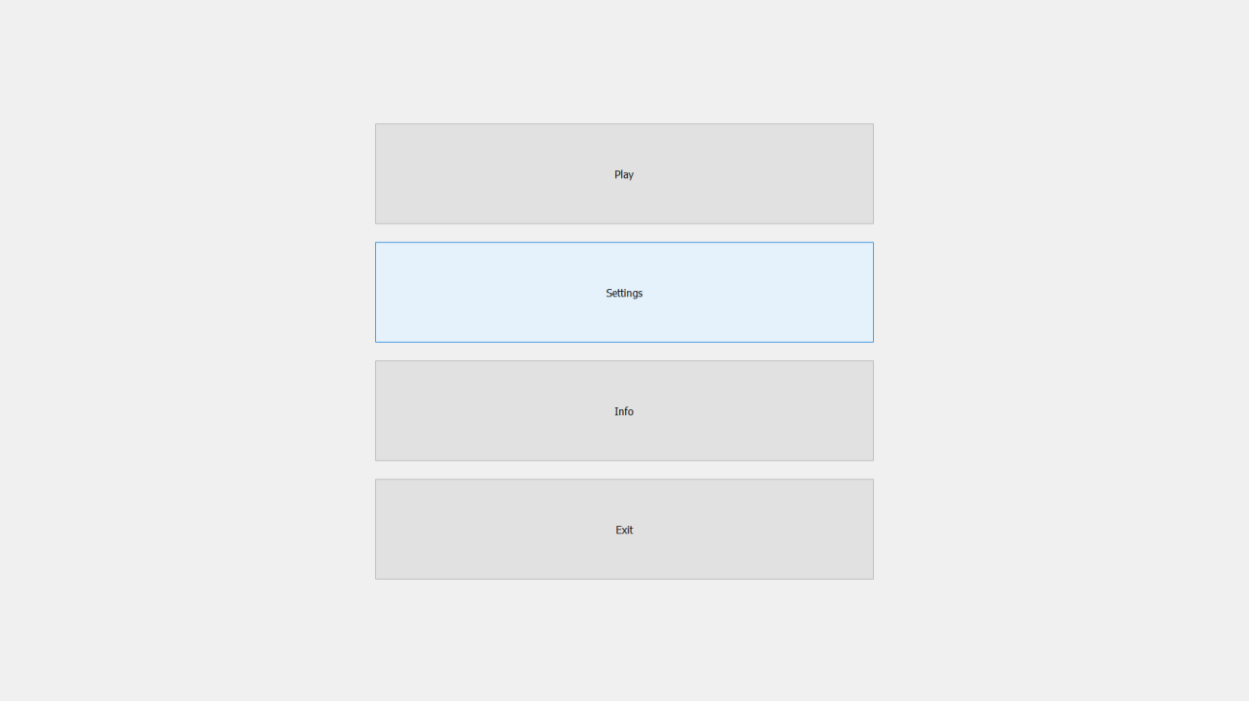
Тут компьютер может прити к симметрии следующим ходом:

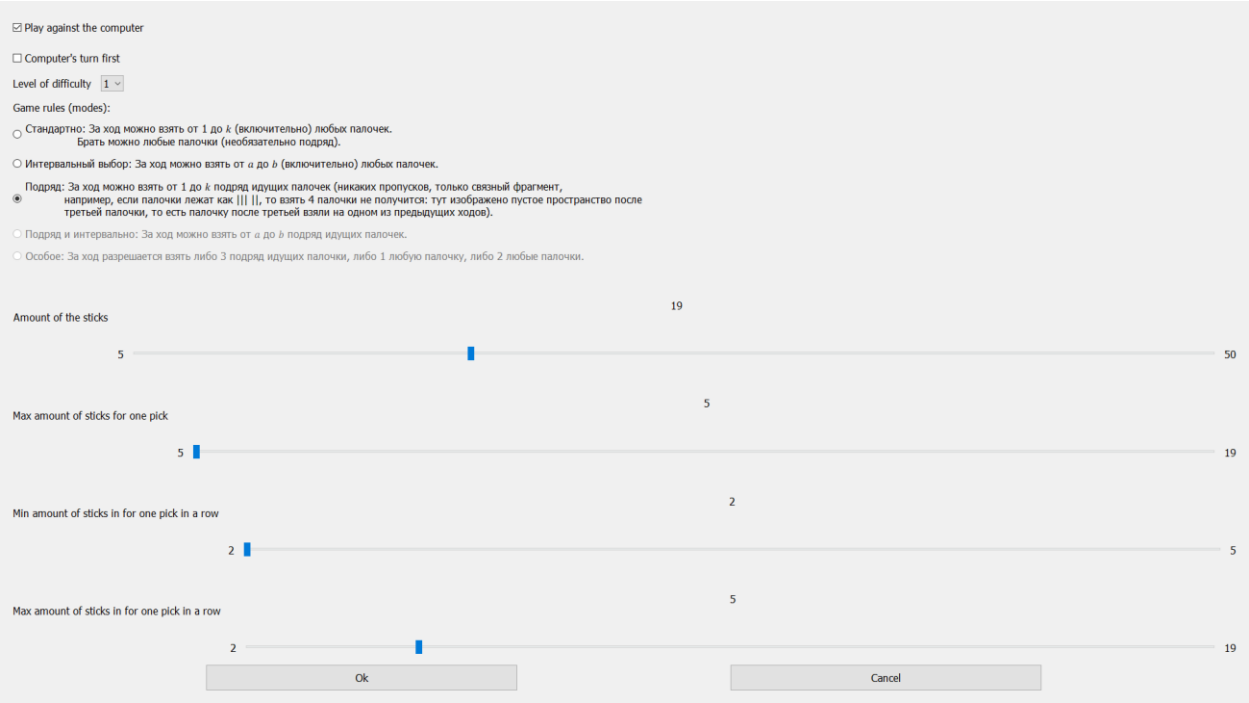
| _ _ _ | | _ _ | | _ _ _ |.

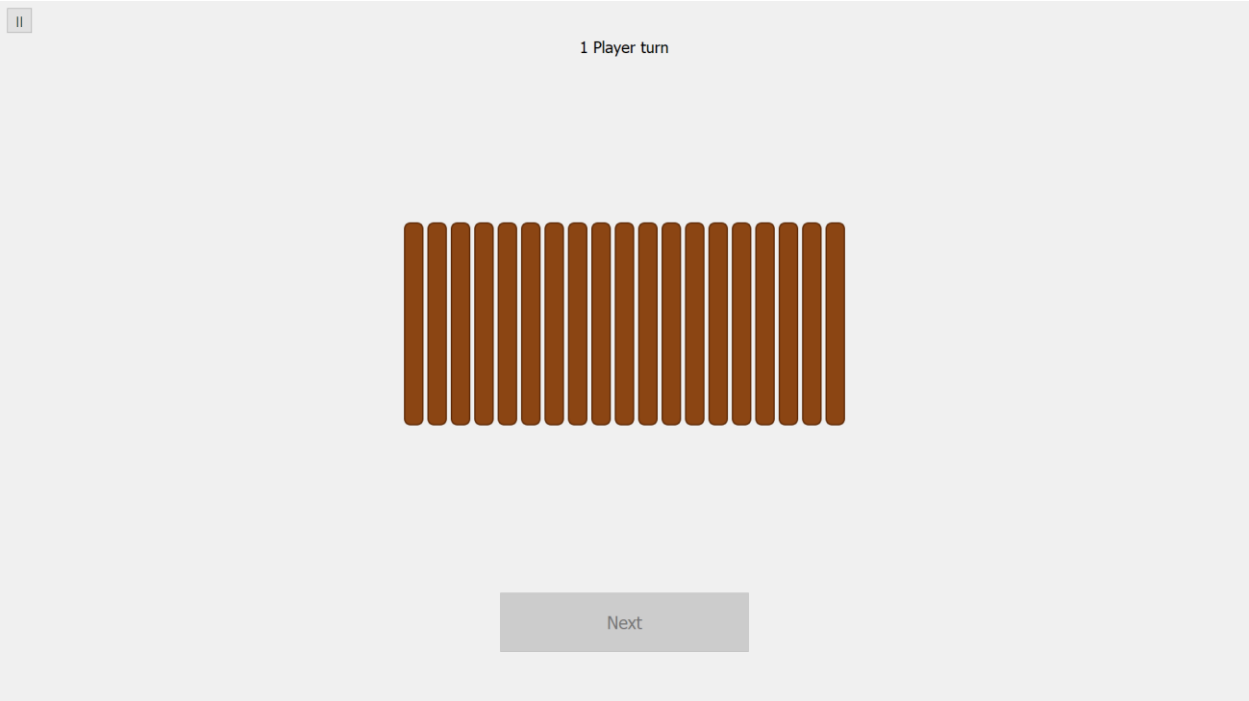
И тогда мы опять вернемся к выигрышной стратегии.

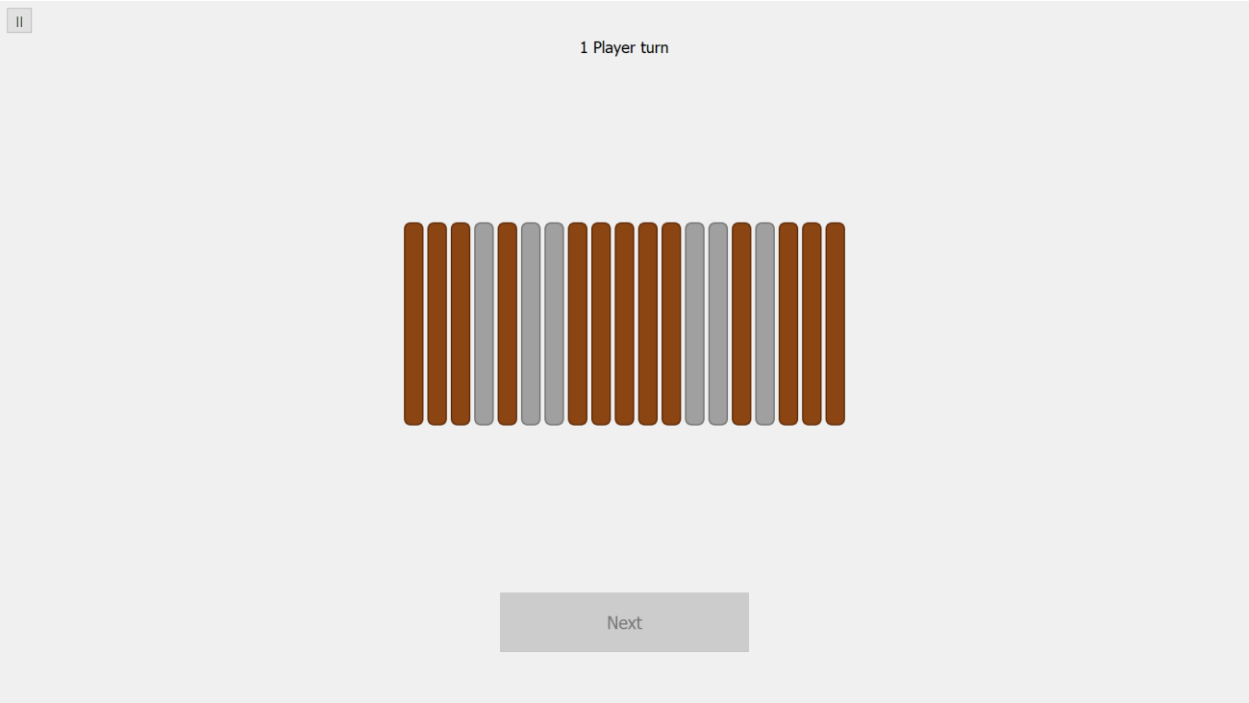
Для разных уровней сложности был использован алгоритм рандомизации, который подстраивается под максимально возможное количество ходов для заданных k, a, b и n, и с определенным шансом делает ход компьютера случайным (в пределах правил)

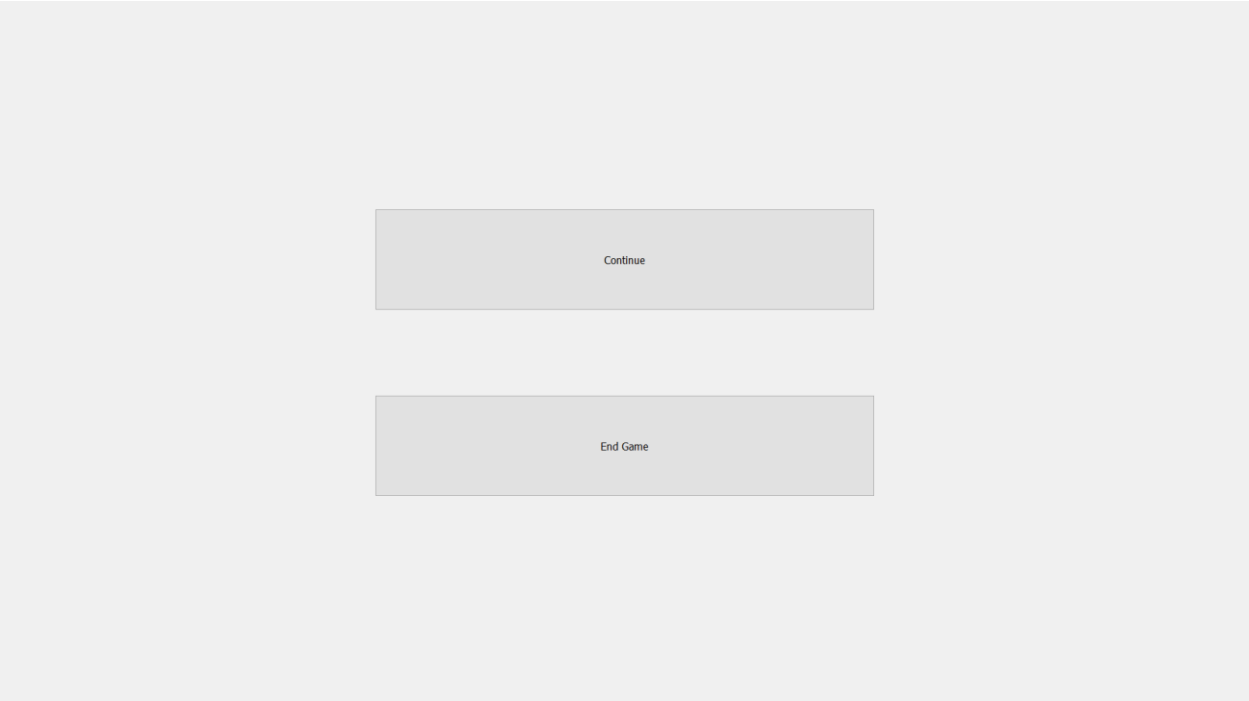
4) Тестирование:




Play against the computer
Computer's turn first
Level of difficulty 1
Game rules (modes):
Стандартно: За ход можно взять от 1 до k (включительно) любых палочек.
Брать можно любые палочки (необязательно подряд).
Интервальный выбор: За ход можно взять от a до b (включительно) любых палочек.
Подряд: За ход можно взять от 1 до k подряд идущих палочек (никаких пропусков, только связный фрагмент,
например, если палочки лежат как ||| ||, то взять 4 палочки не получится: тут изображено пустое пространство после
третьей палочки, то есть палочку после третьей взяли на одном из предыдущих ходов).
Подряд и интервально: За ход можно взять от a до b подряд идущих палочек.
Особое: За ход разрешается взять либо 3 подряд идущих палочки, либо 1 любую палочку, либо 2 любые палочки.
Amount of the sticks
19
5
50
Max amount of sticks for one pick
5
5
19
Min amount of sticks in for one pick in a row
2
2
5
Max amount of sticks in for one pick in a row
5
2
19
Ok
Cancel


||
1 Player turn
Next


II
1 Player turn
Next


Continue
End Game

5) Итоги:

В результате было разработано приложение для тренировки в игру в палочки.

Были реализованы:

Возможность игры против компьютера (для нек-рых режимов);

Возможность выбора очередности ходов;

Первый вариант правил, игра с компьютером, уровни сложности;

Второй вариант правил, игра с компьютером, уровни сложности;

Третий вариант правил, игра с компьютером на первом уровне сложности (при выборе остальных для данного режима компьютер будет играть идентично);

Возможности выбора параметров k, a, b и n с помощью слайдеров;

Возможность остановки/продолжения в процессе игры;

Адаптивность интерфейса под разные размеры экрана.